code/nndl/optim.py

```python
import numpy as np

"""
This code was originally written for CS 231n at Stanford University
(cs231n.stanford.edu).  It has been modified in various areas for use in the
ECE 239AS class at UCLA.  This includes the descriptions of what code to
implement as well as some slight potential changes in variable names to be
consistent with class nomenclature.  We thank Justin Johnson & Serena Yeung for
permission to use this code.  To see the original version, please visit
cs231n.stanford.edu.
"""

"""
This file implements various first-order update rules that are commonly used for
training neural networks. Each update rule accepts current weights and the
gradient of the loss with respect to those weights and produces the next set of
weights. Each update rule has the same interface:

def update(w, dw, config=None):

Inputs:
  - w: A numpy array giving the current weights.
  - dw: A numpy array of the same shape as w giving the gradient of the
    loss with respect to w.
  - config: A dictionary containing hyperparameter values such as learning rate,
    momentum, etc. If the update rule requires caching values over many
    iterations, then config will also hold these cached values.

Returns:
  - next_w: The next point after the update.
  - config: The config dictionary to be passed to the next iteration of the
    update rule.

NOTE: For most update rules, the default learning rate will probably not perform
well; however the default values of the other hyperparameters should work well
for a variety of different problems.

For efficiency, update rules may perform in-place updates, mutating w and
setting next_w equal to w.
"""


def sgd(w, dw, config=None):
    """
    Performs vanilla stochastic gradient descent.

    config format:
    - learning_rate: Scalar learning rate.
    """
    if config is None:
        config = {}
```

```python
    config.setdefault("learning_rate", 1e-2)

    w -= config["learning_rate"] * dw
    return w, config


def sgd_momentum(w, dw, config=None):
    """
    Performs stochastic gradient descent with momentum.

    config format:
    - learning_rate: Scalar learning rate.
    - momentum: Scalar between 0 and 1 giving the momentum value.
      Setting momentum = 0 reduces to sgd.
    - velocity: A numpy array of the same shape as w and dw used to store a moving
      average of the gradients.
    """
    if config is None:
        config = {}
    config.setdefault("learning_rate", 1e-2)
    config.setdefault("momentum", 0.9)  # set momentum to 0.9 if it wasn't there
    v = config.get("velocity", np.zeros_like(w))  # gets velocity, else sets it to zero.

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the momentum update formula.  Return the updated weights
    #   as next_w, and the updated velocity as v.
    # ================================================================ #

    # Calculate the current velocity
    v = config['momentum'] * v - config['learning_rate'] * dw

    # Update the weights using the velocity
    next_w = w + v

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    config["velocity"] = v

    return next_w, config


def sgd_nesterov_momentum(w, dw, config=None):
    """
    Performs stochastic gradient descent with Nesterov momentum.

    config format:
    - learning_rate: Scalar learning rate.
    - momentum: Scalar between 0 and 1 giving the momentum value.
      Setting momentum = 0 reduces to sgd.
    - velocity: A numpy array of the same shape as w and dw used to store a moving
```

```
      average of the gradients.
    """
    if config is None:
        config = {}
    config.setdefault("learning_rate", 1e-2)
    config.setdefault("momentum", 0.9)  # set momentum to 0.9 if it wasn't there
    v = config.get("velocity", np.zeros_like(w))  # gets velocity, else sets it to zero.

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the momentum update formula.  Return the updated weights
    #   as next_w, and the updated velocity as v.
    # ================================================================ #

    # Store the previous velocity
    v_prev = v

    # Update the current velocity
    v = config['momentum'] * v - config['learning_rate'] * dw

    # Apply the Nesterov momentum update rule (lookahead adjustment)
    next_w = w + v + config['momentum'] * (v - v_prev)

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    config["velocity"] = v

    return next_w, config


def rmsprop(w, dw, config=None):
    """
    Uses the RMSProp update rule, which uses a moving average of squared gradient
    values to set adaptive per-parameter learning rates.

    config format:
    - learning_rate: Scalar learning rate.
    - decay_rate: Scalar between 0 and 1 giving the decay rate for the squared
      gradient cache.
    - epsilon: Small scalar used for smoothing to avoid dividing by zero.
    - beta: Moving average of second moments of gradients.
    """
    if config is None:
        config = {}
    config.setdefault("learning_rate", 1e-2)
    config.setdefault("decay_rate", 0.99)
    config.setdefault("epsilon", 1e-8)
    config.setdefault("a", np.zeros_like(w))

    # ================================================================ #
    # YOUR CODE HERE:
```

```
    #   Implement RMSProp.  Store the next value of w as next_w.  You need
    #   to also store in config['a'] the moving average of the second
    #   moment gradients, so they can be used for future gradients. Concretely,
    #   config['a'] corresponds to "a" in the lecture notes.
    # ================================================================ #

    # Update the moving average of the squared gradients (second moment)
    config['a'] = config['decay_rate'] * config['a'] + (1 - config['decay_rate']) * (dw ** 2)

    # Update the weights by dividing the learning rate by the square root of the moving average
    next_w = w - config['learning_rate'] * dw / (np.sqrt(config['a']) + config['epsilon'])

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    return next_w, config


def adam(w, dw, config=None):
    """
    Uses the Adam update rule, which incorporates moving averages of both the
    gradient and its square and a bias correction term.

    config format:
    - learning_rate: Scalar learning rate.
    - beta1: Decay rate for moving average of first moment of gradient.
    - beta2: Decay rate for moving average of second moment of gradient.
    - epsilon: Small scalar used for smoothing to avoid dividing by zero.
    - m: Moving average of gradient.
    - v: Moving average of squared gradient.
    - t: Iteration number.
    """
    if config is None:
        config = {}
    config.setdefault("learning_rate", 1e-3)
    config.setdefault("beta1", 0.9)
    config.setdefault("beta2", 0.999)
    config.setdefault("epsilon", 1e-8)
    config.setdefault("v", np.zeros_like(w))
    config.setdefault("a", np.zeros_like(w))
    config.setdefault("t", 0)

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement Adam.  Store the next value of w as next_w.  You need
    #   to also store in config['a'] the moving average of the second
    #   moment gradients, and in config['v'] the moving average of the
    #   first moments.  Finally, store in config['t'] the increasing time.
    # ================================================================ #

```

```python
    # Increment the iteration counter
    config['t'] += 1

    # Update the moving average of the first moment (gradient)
    config['v'] = config['beta1'] * config['v'] + (1 - config['beta1']) * dw

    # Update the moving average of the second moment (squared gradient)
    config['a'] = config['beta2'] * config['a'] + (1 - config['beta2']) * (dw ** 2)

    # Compute bias-corrected first and second moments to mitigate initialization bias
    v_hat = config['v'] / (1 - config['beta1'] ** config['t'])
    a_hat = config['a'] / (1 - config['beta2'] ** config['t'])

    # Update the weights using the Adam formula
    next_w = w - config['learning_rate'] * v_hat / (np.sqrt(a_hat) + config['epsilon'])

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    return next_w, config

```